```
[ubuntu@ip-172-31-89-150:~$ sudo docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Ports}}\t{{.Status}}"
CONTAINER ID   NAMES                       PORTS                                                                  STATUS
abac64c0c8a4   ums-add-service_kafka_1     0.0.0.0:9092->9092/tcp, :::9092->9092/tcp                              Up 12 minutes
f4aa72bcec4a   ums-add-service_ums_1       8080/tcp                                                               Up 12 minutes
b8f8e9d941b5   ums-add-service_zookeeper_1 2888/tcp, 3888/tcp, 0.0.0.0:2181->2181/tcp, :::2181->2181/tcp, 8080/tcp   Up 12 minutes
```

Figure 1: Running containers

```
[root@f4aa72bcec4a:/# curl -H "Content-Type: application/json" -d '{"name":"Kr]
 ishna Mohan", "phone":9731423166}' http://localhost:8080/user
 {"name":"Krishna Mohan","phone":9731423166,"since":"2022-05-15T18:03:40.111+0
 0:00"}root@f4aa72bcec4a:/#
```

Figure 2: Posting a new user to the *AddService*

```
[ubuntu@ip-172-31-89-150:~$ sudo docker exec -it abac64c0c8a4 bash                ]
[I have no name!@abac64c0c8a4:/$ cd /opt/bitnami/kafka/bin/                       ]
[I have no name!@abac64c0c8a4:/opt/bitnami/kafka/bin$ ./kafka-console-consumer.sh ]
 --bootstrap-server localhost:9092 --topic com.glarimy.ums.user.add --from-beginning
 {"name":"Krishna Mohan","phone":9731423166,"since":1652637749252}
 {"name":"Krishna Mohan","phone":9731423166,"since":1652637820111}
```

Figure 3: Starting a consumer

It gives an output like Figure 2.

You can exit the *bash* terminal and verify the logs of the container using the following command:

```
sudo docker logs f4aa72bcec4a
```

You can see that the *AddService* is processing the POST requests by saving the user data and also by sending the notifications to Kafka.

As the last activity, let us verify that Kafka is indeed receiving the notifications from the *AddService*. First, identify the container ID of the Kafka and log into the container:

```
sudo docker exec -it abac64c0c8a4 bash
```

Run the following command to start a consumer that listens to the topic on which the *AddService* was writing messages.

```
/opt/bitnami/kafka/bin//kafka-console-consumer.sh
--bootstrap-server localhost:9092 --topic com.glarimy.ums.
user.add --from-beginning
```

An output similar to Figure 3 can be observed.

Once the job is done, the following command can bring down all the services at once:

```
sudo docker-compose down
```

What did we do?

We just accomplished two objectives. We were able to create a network and orchestrate multiple Docker containers. We were also able to connect *AddService* and Kafka cluster on Docker. The beauty of this architecture is that the services are not actually aware of each other. It is only through the configuration that we are connecting them. And the producers and consumers of Kafka do not block each other.

In the next several parts, we will develop and deploy the *FindService* and *SearchService* on the Python and Node environments, orchestrate containers on different machines using Kubernetes, scale them, and also develop the *JournalService* as a Kafka consumer. END

By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.